DEEP FAKE REAL-TIME DETECTION

Major project report submitted to CUSAT in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE & ENGINEERING

Submitted by,

AJAY P REJI (20220507)

ARJUN JOSHI M (20220517)

ARJUN V M (20220519)

DOMINIC JOHNSON (20220558)

SAHUL M S (20420546)

Under the guidance of

Dr. Preetha Mathew K,

Head of Department, Division of CSE

Mrs. Aswathy V Shaji,

Assistant Professor, Division of CSE

Mrs. Anitha Mary Chacko,

Assistant Professor, Division of CSE

Division of Computer Science & Engineering

COCHIN UNIVERSITY COLLEGE OF ENGINEERING KUTTANAD

ALAPPUZHA

APRIL 2024

COCHIN UNIVERSITY COLLEGE OF ENGINEERING KUTTANAD

ALAPPUZHA

BONAFIDE CERTIFICATE

This is to certify that the major project certified **Deep Fake Real-Time Detection** is a bonafide report of major project done by **ARJUN VM(20220519), AJAY P REJI920220507), ARJUN JOSHI M(20220517), DOMINIC JOHNSON(20220558), SAHUL MS(20220546)** towards the partial fulfillment of the requirements of the degree of B.Tech in COMPUTER SCIENCE AND ENGINEERING of COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY

Mrs. Aswathy Shaji
Assistant Professor
Division of CSE

Dr.Preetha Mathew K
Head of Department
Division of CSE

Mrs. Anitha Mary Chacko
Assistant Professor
Division of CSE

Place: Pulincunnu

Date: 22-04-2024

ACKNOWLEDGMENT

We would like to express our special thanks to Dr. Preetha Mathew, Mrs. Anitha Mary Chacko and Mrs. Aswathy V Shaji for their guidance, constant supervision, and encouragement. Efforts have been taken by us in this project however it would not have been possible without the kind of support and help of many individuals and organizations. Thanks to our respected principal, Dr. Josephkutty Jacob sir, for the facilities provided by him during the preparation of this report. We also express our gratitude towards all the staff members of the Computer Science and Engineering department and faculties of CUCEK for their guidance. We would like to extend our sincere gratitude to Dr. Preetha Mathew, Head of the Department for giving us innovative suggestions, timely advice, correction, and suggestions during this endeavor.

ABSTRACT

In an age overshadowed by the looming threat of hyper-realistic deep fake videos, propelled by the relentless march of computational advancements, the urgency for inventive and viable solutions has reached unprecedented levels. These artificially generated videos wield the potential to sow seeds of political turmoil, propagate false narratives of terrorism, and enable various forms of exploitation, demanding swift and decisive action. This project introduces an innovative method poised to confront this looming menace head-on within real-time video contexts. Leveraging a combination of Res-Net, PCA (Principal Component Analysis), and SVM (Support Vector Machine), our approach adeptly discerns between authentic live videos and AI-generated deep fakes, with a keen emphasis on identifying replacement and re-enactment techniques. Through rigorous evaluation on real-time video dataset Deepfake Detection Challenge [1], our solution demonstrates its practical efficacy, ensuring prompt detection and response to uphold the integrity of live video content. This system represents a significant leap forward in combating the nefarious applications of deep fake technology in dynamic, real-world scenarios, offering a robust defense against the rampant spread of counterfeit videos.

TABLE OF CONTENTS

CHAPTER 1
INTRODUCTION

1. INTRODUCTION

In an era marked by the escalating threat of hyper-realistic deep fake videos, fueled by rapid advancements in computational technology, the need for innovative and practical solutions has never been more pressing. These AI-generated videos have the potential to incite political unrest, spread false narratives of terrorism, and facilitate various forms of exploitation, demanding immediate attention. This research paper introduces a cutting-edge method that employs Artificial Intelligence (AI) to combat this menace in real-time video scenarios. By integrating a Res-Net, PCA(Principal Component Analysis) & SVM(Support Vector Machine), our approach effectively distinguishes live, authentic videos from AI-generated deep fakes, with a particular focus on replacement and re-enactment deep fake techniques. Rigorous evaluation on real-time video data underscores the practicality of our solution, ensuring timely detection and response to safeguard the integrity of live video content. This research represents a significant stride in countering the malicious use of deep fake technology in dynamic, real-time applications, offering a robust defense against the proliferation of fake videos.

1.1 AIM

Our aim is to develop an advanced AI solution for real-time detection and differentiation of hyper realistic deep fake videos from authentic content. This project seeks to effectively combat the rising threat posed by deep fakes, which have the potential to incite political unrest, fake terrorism, and exploitation. We intend to leverage AI to offer immediate and accurate classification of live videos as either deep fakes or real content, ultimately providing a highly responsive and reliable defense against the malicious use of deep fake technology in dynamic, real-time applications.

1.2 OBJECTIVE

The main goal is to detect deep fake videos in real-time, ensuring the authenticity of live content and countering their malicious use in dynamic applications.

1.3 PURPOSE

In the ever-evolving landscape of social media, our project stands as a bulwark against the menacing tide of hyper-realistic deep fake videos. Fuelled by the unprecedented computational process, these AI generated forgeries have become potent tools, capable of sowing discord, fabricating events, and ruining lives. Our mission is clear: to harness the very power of Artificial Intelligence that enables these deceptions. In the heart of our research lies a cutting-edge fusion of Res-Net, PCA(Principal Component Analysis) & SVM(Support Vector Machine), meticulously calibrated to discern the subtlest nuances between live, genuine videos and their deceptive counterparts. Through an exhaustive training regimen encompassing diverse datasets like (DFDC)Deep fake Detection Challenge, our project epitomizes a relentless pursuit of accuracy and immediacy. By pioneering a real-time detection system, we are not just creating a shield against malicious deep fakes; we are shaping a new frontier of trust and authenticity in the digital realm.

1.4 SCOPE

There are many tools available for creating the deep fakes, but for deep fake detection there is hardly any tool available. Our approach for detecting the deep fakes will be great contribution in avoiding the percolation of the deep fakes over the World Wide Web. We will be providing a web-based platform for the user to upload the video and classify it as fake or real. This project can be scaled up from developing a web-based platform to a browser plugin for automatic deep fake detection’s. Even big application like WhatsApp, Facebook can integrate this project with their application for easy pre-detection of deep fakes before sending to another user. A description of the software with Size of input, bounds on input, input validation, input dependency, i/o state diagram, Major inputs, and outputs are described without regard to implementation detail.

CHAPTER 2
OVERVIEW

2. OVERVIEW

2.1 PROJECT OVERVIEW

Planning:

The project planning phase involves identifying the critical issue of deep fake technology and formulating a strategy to address it. Recognizing the need for a solution to distinguish between authentic and manipulated videos, the project sets the goal of leveraging the inherent limitations of deep fake creation tools. The planning phase also entails the decision to use neural network tools like GAN and Auto Encoders, with a focus on ResNet Convolution Neural Networks for detecting artefacts.

Analysis:

In the analysis phase, the project delves into the intricacies of deep fake technology, understanding how GANs and Auto Encoders create realistic videos. The identification of distinguishable artifacts left by these tools becomes a key insight. The analysis establishes the foundation for utilizing Res-Net CNNs to capture these artifacts effectively. The examination of existing deep fake datasets, including (DFDC) Deep fake Detection Challenge, is undertaken to ensure a comprehensive understanding of the problem space.

Design:

The design phase translates the insights from the analysis into a systematic approach. The project envisions a methodology where Res-Net CNNs extract frame-level features, PCA reduce the dimensionality of the extracted features from the Res-Net & SVM for classification. This design ensures the incorporation of the identified artifacts into the classification process. The strategy includes a deliberate combination of diverse datasets during training to enhance the model's capability to discern different features and improve its performance on real-time data.

Prototype:

The prototype phase involves the creation of a working model that embodies the designed approach. Neural network tools such as GAN and Auto Encoders are implemented to simulate the deep fake creation process. Res-Next CNNs are employed to capture artifacts, and the SVM is developed for video classification. This prototype serves as an initial representation of the envisioned solution and provides a tangible basis for testing and refinement before full-scale implementation.

Implementation:

The implementation phase marks the transition from the prototype to a fully functional system. The project utilizes the chosen neural network tools and architectures in a practical setting. Res-Net CNNs are integrated to extract frame-level features, the PCA and SVM are trained on a diverse dataset, including videos from deep fake Detection Challenge. The system aims to effectively classify videos as either manipulated (deep fake) or authentic. Real-time evaluation becomes a focal point, ensuring the model's applicability in dynamic

scenarios and contributing to the broader community's well-being by preventing the potential misuse of deep fake technology.

2.2 PROJECT FEASIBILITY REPORT

Our primary goal is to make informed decisions about the viability and potential success of your deep fake detection project. It's important to periodically revisit these feasibility studies as the project progresses to account for any changes in circumstances or requirements.

Here are different aspects of feasibility that we should consider:

1. Technical Feasibility:

- Data Availability: We've meticulously reviewed available datasets suitable for our deep fake detection model. Notable datasets include Face-Forensic++, Deep fake Detection Challenge, and Celeb-DF. The data collection phase is actively underway, involving scrupulous inspection to ensure high quality and diverse data for effective model training and testing.
- Technology Stack: Our technology stack is tailored to our academic environment, incorporating GANs, Auto encoders, and Res-Net Convolution Neural Networks based on available academic resources.

2. Operational Feasibility:

- Resource Requirements: We've estimated the computational resources, software tools, and skill sets required for our 5-member student team. Resource allocation is proceeding in alignment with the academic environment, ensuring each team member is actively engaged and contributing to their respective roles.
- Integration with Existing Systems: To minimize disruption, our deep fake detection system is being designed for seamless integration within our academic environment.

3. Economic Feasibility:

- Cost Analysis: Operating within the constraints, a detailed cost analysis covers academic resources, minimal out of-pocket expenses, and any potential funding sources. We are managing within a student friendly budget, securing funds through available academic channels and keeping expenditures within reasonable bounds.
- Return on Investment (ROI): Given our student context, the ROI is evaluated primarily in terms of academic benefits and learning outcomes. Regular tracking of academic achievements and learning experience servers as our primary metric for return, with a focus on skill development and knowledge acquisition.

4. Legal and Ethical Feasibility:

- Compliance with Regulations: Legal considerations are rigorously reviewed based on academic guidelines and ethical standards relevant to our university context. We prioritize adherence to academic regulations and ethical guidelines, with regular reviews to ensure ongoing compliance.
- Ethical Implications: Potential biases in our model are carefully considered, and ethical guidelines are established in accordance with academic standards. Ethical considerations will be revisited during model training, with the guidance of academic advisors shaping our decisions and approaches.

5. Schedule Feasibility:

- Timeline: A comprehensive project timeline has been developed, factoring in academic semester schedules, key milestones, and deliverables. Milestones are actively tracked, with flexibility to accommodate academic demands and ensure project progress within the academic calendar.
- Dependencies: External dependencies, such as access to specific academic resources, have been identified, and contingency plans are in place to address any academic-related dependencies. Regular communication with academic advisors ensures timely resolution of dependencies, allowing for smooth project progression.

6. Market Feasibility:

- User Acceptance: Initial feedback is actively sought from academic peers, professors, and relevant stakeholders to understand academic needs, preferences, and potential user acceptance. Continuous feedback loops are established, and academic user acceptance will guide iterative system refinement throughout the development process.
- Competitive Analysis: Our team has undertaken an extensive review of academic literature and existing research projects to position our project within the academic landscape. Regular literature reviews and academic analyses will guide our project's academic positioning, with an emphasis on contributing to the existing body of knowledge.

CHAPTER 3
PROBLEM SPECIFICATIONS

3. PROBLEM SPECIFICATION

3.1 LITERATURE SURVEY

- Face Warping Artifacts [3] used the approach to detect artifacts by comparing the generated face areas and their surrounding regions with a dedicated Convolutional Neural Network model. In this work there were two-fold of Face Artifacts. Their method is based on the observations that current deepfake algorithm can only generate images of limited resolutions, which are then needed to be further transformed to match the faces to be replaced in the source video. Their method has not considered the temporal analysis of the frames.

- Detection by Eye Blinking [4] describes a new method for detecting the deepfakes by the eye blinking as a crucial parameter leading to classification of the videos as deepfake or pristine. The Long-term Recurrent Convolution Network (LRCN) was used for temporal analysis of the cropped frames of eye blinking. As today the deepfake generation algorithms have become so powerful that lack of eye blinking can not be the only clue for detection of the deepfakes. There must be certain other parameters must be considered for the detection of deepfakes like teeth enchantment, wrinkles on faces, wrong placement of eyebrows etc.

- Capsule networks to detect forged images and videos [5] uses a method that uses a capsule network to detect forged, manipulated images and videos in different scenarios, like replay attack detection and computer-generated video detection. In their method, they have used random noise in the training phase which is not a good option. Still the model performed beneficial in their dataset but may fail on real time data due to noise in training. Our method is proposed to be trained on noiseless and real time datasets.

- Recurrent Neural Network (RNN) [6] for deepfake detection used the approach of using RNN for sequential processing of the frames along with ImageNet pre-trained model. Their process used the HOHO [7] dataset consisting of just 600 videos. Their dataset consists small number of videos and same type of videos, which may not perform very well on the real time data. We will be training out model on large number of Realtime data.

- Synthetic Portrait Videos using Biological Signals [8] approach extract biological signals from facial regions on pristine and deepfake portrait video pairs. Applied transformations to compute the spatial coherence and temporal consistency, capture the signal characteristics in feature vector and photo plethysmography (PPG) maps, and further train a probabilistic Support Vector Machine (SVM) and a Convolutional Neural Network (CNN). Then, the average of authenticity probabilities is used to classify whether the video is a deepfake or a pristine.

- Fake Catcher detects fake content with high accuracy, independent of the generator, content, resolution, and quality of the video. Due to lack of discriminator leading to the loss in their findings

to preserve biological signals, formulating a differentiable loss function that follows the proposed signal processing steps is not straight forward process.

3.2 PROBLEM STATEMENT

While advancements in AI have made it easier to create convincing deep fakes, detecting these manipulated videos is a pressing challenge. The proliferation of deep fakes poses serious risks, including the spread of misinformation and the exploitation of individuals. In response, we aim to develop an effective deep learning-based solution for detecting fake faces in videos, with a focus on identifying deep fakes accurately. Our approach utilizes a combination of image processing, feature extraction, and machine learning techniques to distinguish between real and manipulated content, thereby mitigating the harmful effects of deep fakes on social media platforms and beyond.

3.3 PROPOSED STATEMENT

Our project is dedicated to addressing the increasingly urgent challenge of detecting real-time live deepfakes, a threat exacerbated by the rapid advancements in AI technology. With the proliferation of deep learning tools, the creation of sophisticated deepfakes has become alarmingly accessible, necessitating a robust defense mechanism to identify and mitigate their harmful effects. Leveraging the code provided above, our proposed system embodies a proactive approach towards this endeavor, employing a comprehensive pipeline for detecting fake faces in videos, with a particular emphasis on real-time detection of deepfakes.

At the core of our system lies a sophisticated framework that encompasses several crucial stages. It begins by collecting live video data, enabling the continuous analysis of potentially manipulated content. Using advanced image processing techniques, including face detection and extraction, the system isolates facial regions for further examination. These extracted faces undergo meticulous pre-processing to ensure optimal input quality, paving the way for accurate feature extraction. Through the utilization of a pre-trained ResNet model and techniques like Principal Component Analysis (PCA), our system extracts high-level features from the pre-processed facial images, which serve as the foundation for subsequent analysis. Trained on a dataset generated using the provided code, a Support Vector Machine (SVM) classifier then distinguishes between authentic and deepfake faces based on these features, achieving high accuracy and reliability in real-time detection scenarios.

CHAPTER 4
PROPOSED METHODOLOGY

4. PROPOSED METHODOLOGY

There are many tools available for creating the Deep fake, but for Real time Deep Fake detection there is hardly any tool available. Our approach for detecting the Real time Deep Fake video will be great contribution in avoiding the percolation of the DF over the world wide web. We will be providing a user interface that takes in the live input video feed of the user and classify it as fake or real. Our system is designed to detect fake faces in videos through a streamlined pipeline of image processing and machine learning techniques. Initially, it collects face images directly from video data, leveraging tools like OpenCV and face_recognition for accurate face extraction. These images are then labeled as either "fake" or "real" and processed for feature extraction.

To differentiate between fake and real faces, we utilize a pre-trained ResNet model to extract essential features from the images. These features undergo optimization via Principal Component Analysis (PCA) to improve computational efficiency while preserving accuracy. The processed features are then inputted into a Support Vector Machine (SVM) classifier, which learns to classify faces based on their authenticity. By leveraging the DeepFake Detection Challenge (DFDC) dataset, the system undergoes rigorous training and evaluation to accurately distinguish between authentic and manipulated content. In real-time, our system can swiftly predict the authenticity of new face images, making it suitable for various applications in security, forensics, and media authentication. By integrating state-of-the-art technologies, our proposed system offers a reliable solution for detecting fake faces in videos.

The proposed methodology for detecting real-time live deepfakes involves a systematic approach that encompasses several detailed steps:

1. Data Collection and Pre-processing:

   - The process initiates with the collection of live video data, which serves as the primary input for real-time analysis. This video data is continuously streamed and captured for immediate processing.
   - Advanced image processing techniques are employed for preprocessing. Initially, the video frames are analyzed to identify and extract facial regions using robust face detection algorithms. Once identified, these facial regions are isolated from the background.
   - Subsequently, the extracted facial images undergo preprocessing steps to enhance their quality and suitability for subsequent analysis. These steps may include resizing the images to a standard size, converting them to a consistent color space, and applying noise reduction techniques to improve clarity.

2. Feature Extraction and Dimensionality Reduction:

   - Feature extraction is a crucial stage where the system extract relevant information from the pre-processed facial images. This is achieved using a pre-trained deep learning model, such as ResNet,

which has been specifically trained on large datasets to recognize facial features effectively.

- After feature extraction, the dimensionality of the extracted features is often quite high, which can lead to increased computational complexity. To address this, dimensionality reduction techniques like Principal Component Analysis (PCA) are applied. PCA compresses the feature space while preserving as much of the original variance as possible, resulting in a more compact representation of the facial features.

3. Model Training and Evaluation:

- The reduced-dimensional feature vectors are used to train a machine learning model, typically a Support Vector Machine (SVM) classifier. This classifier learns to distinguish between authentic and manipulated facial images based on the extracted features.
- The training process involves feeding labelled data into the SVM classifier and adjusting its parameters to minimize classification errors. This step is crucial for ensuring that the classifier can accurately differentiate between real and deepfake faces.
- Once trained, the performance of the SVM classifier is evaluated using a variety of metrics, including accuracy, precision, recall, and F1 score. Additionally, confusion matrix analysis provides insights into the classifier's ability to correctly classify both real and deepfake faces.

4. Real-time Prediction and Analysis:

- In real-time scenarios, the trained SVM classifier is deployed to predict the authenticity of faces detected in live video streams. As new video frames are captured, the classifier rapidly analyzes each frame to determine whether it contains a real or deepfake face.
- The system continuously monitors the video stream, flagging any instances of potential deepfake content. Visualizations such as heatmaps and Receiver Operating Characteristic (ROC) curves may be used to visualize and interpret the classifier's performance in real-time detection scenarios.

By meticulously following this detailed methodology, our system aims to effectively detect and mitigate the dissemination of real-time live deepfakes, thereby safeguarding against their potentially harmful consequences.

CHAPTER 5
REQUIREMENT ANALYSIS

5. REQUIREMENT ANALYSIS

5.1 FUNCTIONAL REQUIREMENTS

1. Live Video Data Collection:
    - The system must continuously collect live video data from various sources, such as webcams or video streams
    - It should support real-time processing of incoming video frames to ensure timely analysis.
2. Facial Region Extraction:
    - The system must accurately detect and extract facial regions from each video frame.
    - It should employ robust face detection algorithms capable of handling variations in lighting, pose, and facial expressions.
3. Pre-processing:
    - Pre-processing steps such as resizing, color space conversion, and noise reduction must be applied to extracted facial images to enhance their quality for subsequent analysis.
4. Feature Extraction and Dimensionality Reduction:
    - The system should utilize a pre-trained deep learning model (e.g., ResNet) to extract relevant facial features from pre-processed images.
    - Techniques like Principal Component Analysis (PCA) must be employed to reduce the dimensionality of the extracted features while preserving their discriminative power.
5. Model Training and Evaluation:
    - It should prepare a labeled dataset comprising both real and deepfake facial images for training the machine learning model.
    - The system must train a Support Vector Machine (SVM) classifier to distinguish between authentic and manipulated facial images.
    - Performance evaluation metrics such as accuracy, precision, recall, and F1 score should be computed to assess the classifier's effectiveness.
6. Real-time Prediction and Analysis:
    - The trained SVM classifier must be deployed to predict the authenticity of faces detected in real-time video streams.
    - The system should continuously monitor the video stream, identifying and flagging potential instance

of deepfake content.

- Visualizations like heatmaps and Receiver Operating Characteristic (ROC) curves may aid in analyzing the classifier's real-time performance.

5.2 NON FUNCTIONAL REQUIREMENTS

1. Accuracy:
   - The system must achieve high accuracy in detecting deepfake content to minimize false positives and negatives.
2. Real-time Processing:
   - It should be capable of processing incoming video frames in real-time to enable timely detection of deepfake content.
3. Robustness:
   - The system should be robust against variations in lighting conditions, facial poses, and facial expressions.
4. Scalability:
   - It should be designed to scale efficiently to handle a large volume of video streams simultaneously.
5. Usability:
   - The system should have a user-friendly interface, making it easy for users to interact with and configure.
6. Security:
   - Measures must be implemented to ensure the security and privacy of the collected video data and analysis results.
7. Resource Efficiency:
   - The system should be optimized for resource efficiency, minimizing computational resources and memory usage during processing.
8. Maintenance and Support:
   - The system should be easy to maintain and provide adequate support for troubleshooting and updates.

5.3 SOFTWARE REQUIREMENTS

1. Operating System: Windows 7+
2. IDE: Google Colab
3. Server-side Script: Python 3.11.5

4. Libraries Used: Open CV, Numpy, Dilib Model: Sklearn

5.4 HARDWARE REQUIREMENTS

1. RAM: Minimum 8 GB
2. Memory: At least
3. Camera: Webcam with 1080p resolution or higher

CHAPTER 6
PROJECT IMPLEMENTATION

6. PROJECT IMPLEMENTATION

6.1 DATASET DETAILS

The proposed project utilizes the DFDC (DeepFake Detection Challenge) dataset as a foundational resource for training and evaluating a real-time live deepfake detection system. Comprising a diverse array of videos, the DFDC dataset encompasses both real and manipulated content, offering annotations that indicate the authenticity of each video. This annotated dataset serves as invaluable ground truth for supervised learning approaches, enabling the development of deepfake detection models capable of accurately discerning between genuine and manipulated footage. With its extensive variability in actors, backgrounds, and facial expressions, the DFDC dataset presents a robust testing ground for the proposed system, challenging it to generalize effectively across various conditions. By adhering to ethical guidelines and legal considerations surrounding deepfake technology, the project ensures responsible usage of the dataset while aiming to deploy a real-time detection system that can swiftly identify and flag deepfake content in streaming video feeds.

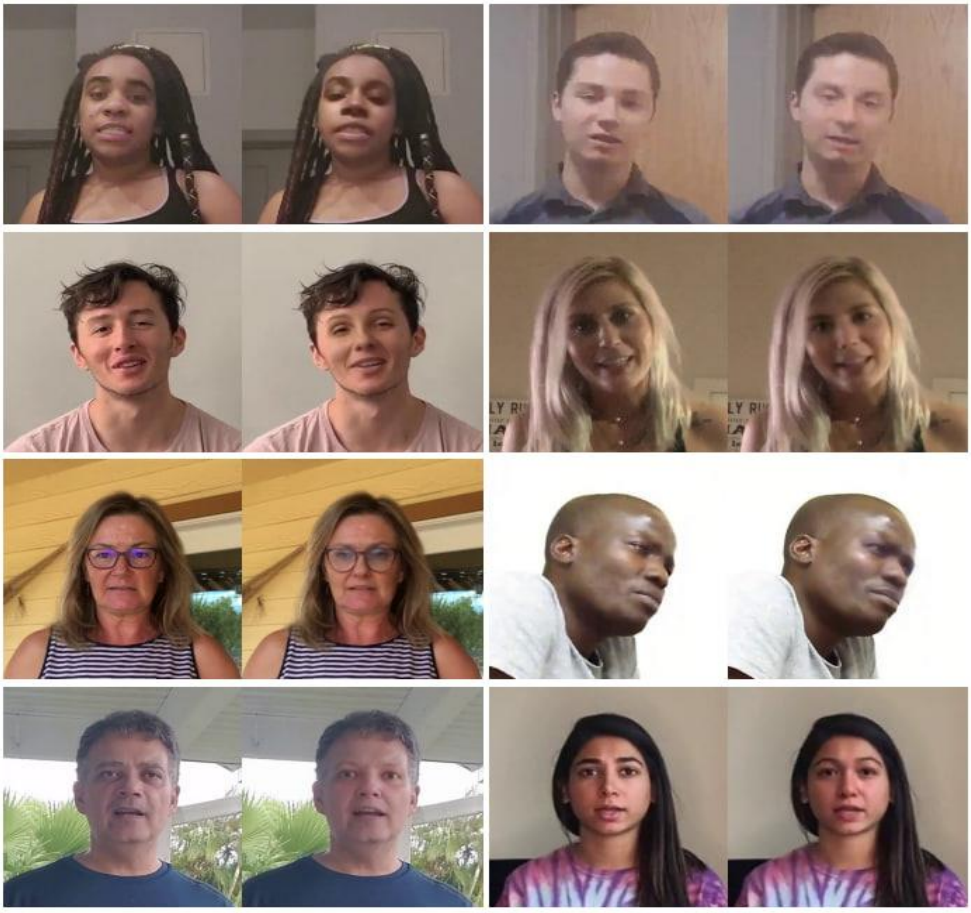


Figure 1: Some examples of DFDC dataset samples

6.2 PREPROCESSING DETAILS

1. Face Detection and Cropping:
   - The system employs face detection algorithms to locate faces within each frame of the video stream.
   - Detected faces are then aligned and cropped to ensure consistent positioning and size using techniques such as the face recognition library.
2. Image Resizing:
   - Following face cropping, the images are resized to a uniform size of 224x224 pixels using the resize function from the skimage.transform module.
   - Resizing the images ensures that all input images have the same dimensions, facilitating consistent processing by the subsequent model layers
3. Color Conversion:
   - The color space of the images is converted from BGR (commonly used in OpenCV) to RGB using the cv2.cvtColor function.
   - Converting the color space to RGB is necessary as many deep learning models, including pre-trained ones like EfficientNet and ResNet, expect input images in RGB format.

6.3 MODEL DETAILS

The model includes the following:

1. Feature Extraction by ResNet:

   The proposed model for real-time live deepfake detection incorporates several critical components to ensure accurate and efficient performance. Firstly, the feature extraction process utilizes a pre-trained ResNet (Residual Neural Network) model. ResNet is renowned for its effectiveness in image classification tasks due to its deep architecture with residual connections. By leveraging ResNet's capabilities, the model can extract high-level features from processed face images. These features encapsulate essential patterns and characteristics necessary for distinguishing between real and deepfake faces. Consequently, employing ResNet for feature extraction enhances the model's ability to discern subtle differences indicative of manipulated content.

   ResNet50, a variant of the Residual Neural Network architecture, is utilized in our model for deep fake detection the provided code snippet for deepfake detection. With 50 layers, it processes input images of size 224x224x3, adhering to the ImageNet-pretrained weights. Omitting fully connected layers (top layers) allows for feature extraction rather than classification. ResNet50's architecture incorporates skip connections, enabling efficient training of deep networks by mitigating the vanishing gradient problem. By leveraging pre-trained weights from ImageNet,

ResNet50 can extract informative features crucial for distinguishing between real and deepfake images, making it a powerful tool for deepfake detection tasks.

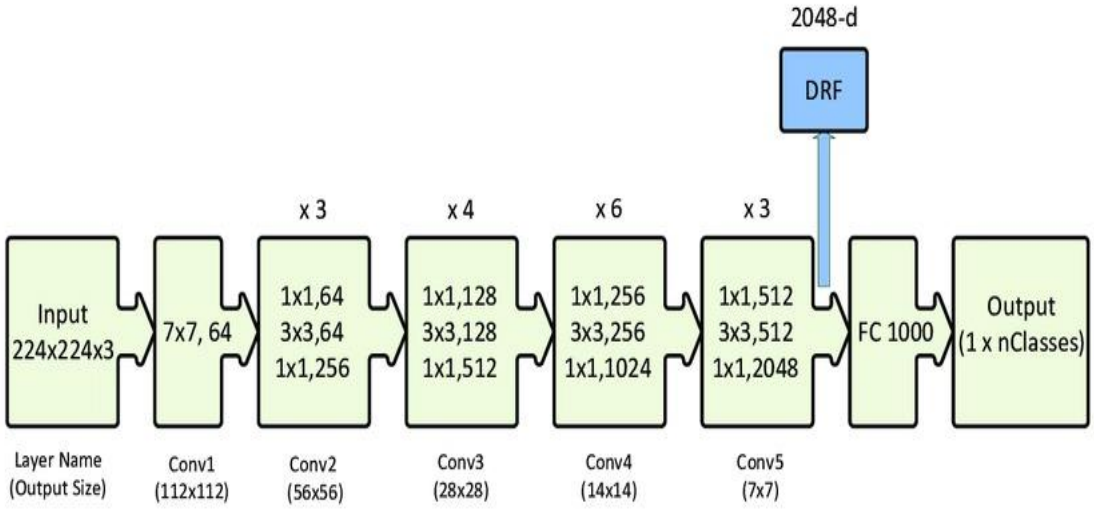


Figure 2: ResNet50 Architecture overview

2. Feature Selection by Principal Component Analysis (PCA) Algorithm:

Following feature extraction, the model employs the Principal Component Analysis (PCA) algorithm for feature selection and dimensionality reduction. PCA is a widely used technique in machine learning for reducing the dimensionality of high-dimensional datasets while preserving most of their variance. By applying PCA to the extracted features, the model effectively reduces computational complexity and mitigates the risk of overfitting. Moreover, PCA enhances the model's generalization ability by focusing on the most informative features, thereby improving its performance in distinguishing between real and deepfake faces.

3. SVM Classifiers for Fast and Accurate Prediction:

In the final classification stage, Support Vector Machine (SVM) classifiers are utilized for making predictions. SVM classifiers are renowned for their ability to offer high accuracy in binary classification tasks, making them well-suited for distinguishing between real and fake faces. Moreover, SVM classifiers are known for their efficiency in both training and prediction phases. This efficiency is particularly crucial for real-time applications, where rapid processing is essential. By employing SVM classifiers, the model achieves a balance between accuracy and speed, ensuring effective real-time detection of deepfake content in streaming video feeds. Overall, the combination of feature extraction by ResNet, feature selection by PCA, and classification by SVM classifiers forms a robust framework for real-time live deepfake detection, capable of accurately identifying and flagging manipulated content with efficiency and reliability.

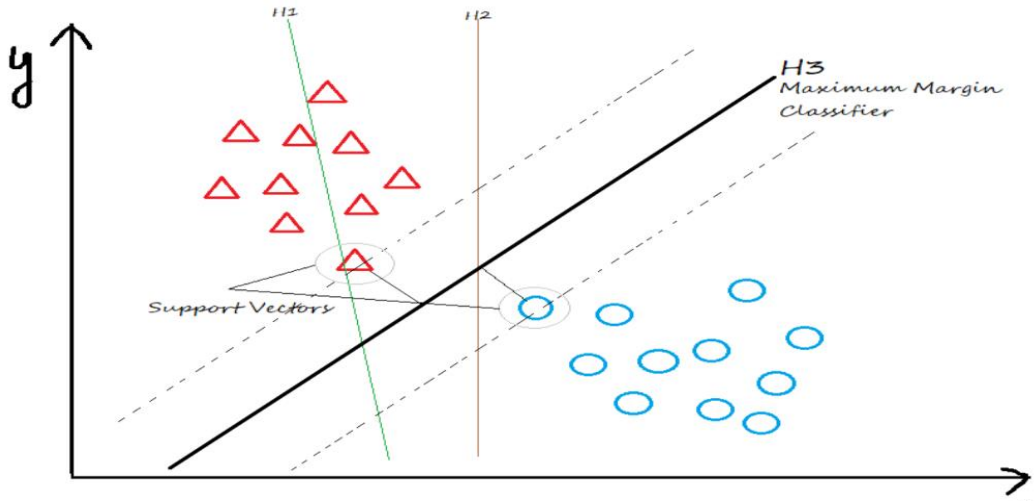


Figure 3: SVM classifier

6.4 MODEL EVALUATION AND PREDICTION

1. Model Evaluation:

- After training the SVM classifier on the extracted features, the model's performance is evaluated using various metrics such as accuracy, F1 score, and confusion matrix.
- Accuracy Score: It measures the proportion of correctly classified instances out of the total instances.
- F1 Score: It provides a balance between precision and recall, particularly useful in imbalanced datasets.
- Confusion Matrix: It visualizes the classifier's performance by showing the number of true positives, true negatives, false positives, and false negatives.
- These evaluation metrics help assess the model's effectiveness in distinguishing between real and fake faces.

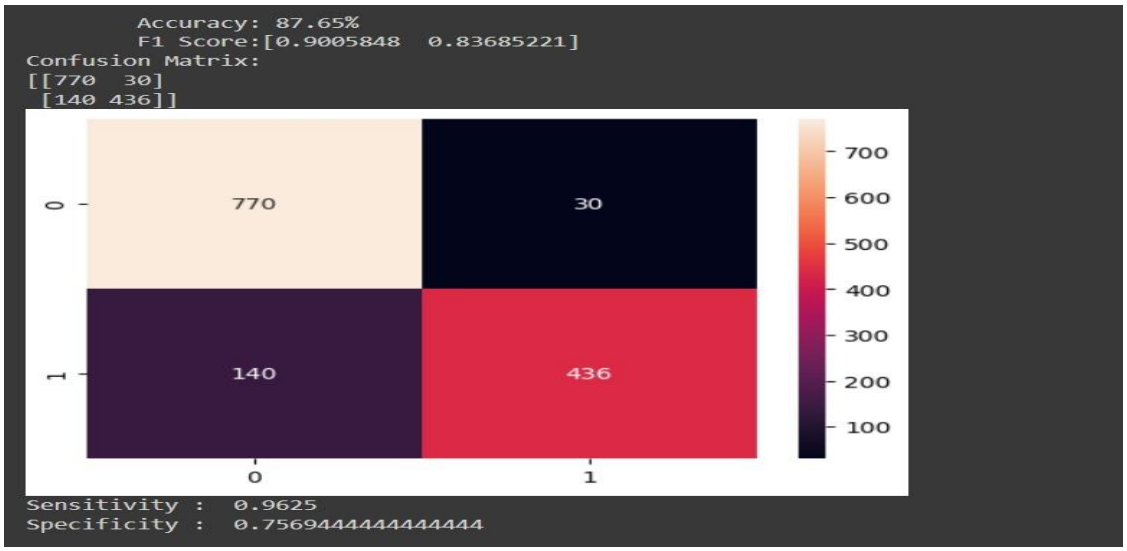


Figure 4: Confusion Matrix

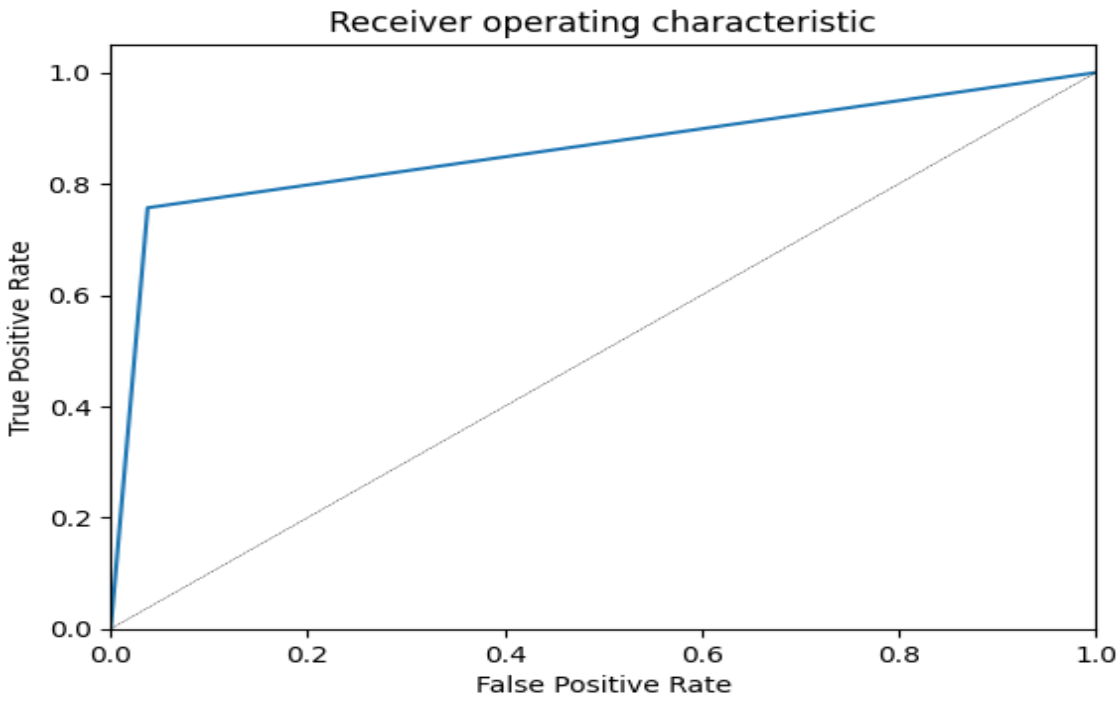


Figure 5: ROC Curve

2. Prediction:

- After evaluating the model, predictions are made on unseen or test data using the trained SVM classifier.
- For each test sample, the extracted features are passed through the SVM classifier, which assigns a label (real or fake) based on the learned decision boundary.
- Predictions are then compared with the ground truth labels to assess the model's accuracy and performance.
- Additionally, individual predictions can be made on specific images using the trained model, where predictions are made for sample images located in the Google Drive folders.
- By analyzing the model's predictions on both test data and specific images, insights can be gained into its effectiveness in detecting deepfake content.

CHAPTER 7
SYSTEM DESIGN

7. SYSTEM DESIGN

7.1 USE CASE DIAGRAM

A use case diagram is a type of behavioral diagram that shows the interactions between a system and its users,or actors. It is used to represent and model the functionality of a system. The main purpose of a use case diagram is to capture the functional requirements of a system. Use case diagrams are typically developed in the early stage of development and often apply use case modeling for the following purposes:

- Specify the context of a system
- Capture the requirements of a system
- Validate a system architecture
- Drive implementation and generate test cases.

The four main components of a use case diagram are actors, the system itself, relationships, and use cases.

- Actors:Actors are the users of a system. They can be people, organizations, or other systems. An actor can be a primary user or a secondary user of a system.
- System:This refers to the system that is being modeled. It is the main focus of the use case diagram. The system’s boundaries are drawn using a rectangle that contains use cases. Place actors outside the system's boundaries.
- Relationships:Relationships show the interaction between the actors and the system. They are often represented by lines or arrows.
- Use Cases:Use cases are the actions that a system performs. They are often represented by ovals.For relationships among use cases, use arrows labelled either "include" or "extend."
    - Extend relationship: The use case is optional and comes after the base use case. It is represented by a dashed arrow in the direction of the base use case with the notation <<extend>>.
    - Include relationship: The use case is mandatory and part of the base use case. It is represented by a dashed arrow in the direction of the included use case with the notation <<include>>.

Figure 6: Use Case Diagram

The use case diagram depicts a deepfake detection system. It shows how a user interacts with the system through different functionalities. Here's a breakdown of the interactions:

Generate Real Video: This use case represents the user uploading a real video to the system for analysis.

Generate Deepfake Video: This use case represents the user uploading a suspected deepfake video to the system for analysis.

Following either of these actions, the system performs a Prediction Real/Fake analysis, which is included in both use cases. This means that no matter what type of video the user uploads (real or deepfake), the system will automatically analyze it to determine its authenticity.

7.2 ACTIVITY DIAGRAM

An activity diagram is another type of behavioral diagram used in UML (Unified Modeling Language) to represent the flow of control within a system or process. Activity diagrams are particularly useful for modeling workflows, business processes, and the sequential steps involved in a system's functionality. Here's an overview of activity diagrams and their components:

Purpose and Usage:

Activity diagrams are employed to model the dynamic aspects of a system, emphasizing the flow of control and data within the system's processes. They are commonly used for:

- Modeling business workflows and processes.
- Describing the steps involved in use cases or scenarios.
- Visualizing the sequence of activities within a system.
- Illustrating the logic and decision points in a process.

Components of Activity Diagrams:

- Activity: An activity represents a specific operation or step within a process. It is depicted as a rounded rectangle with the name of the activity inside.
- Initial Node: The initial node indicates where the process begins. It's represented by a filled circle.
- Final Node: The final node signifies the end of the process or workflow. It's depicted by a circle with a dot inside.
- Action State: An action state represents a discrete operation within the system. It's shown as a rectangle with rounded corners.
- Decision Node (Decision Point): A decision node represents a point in the process where a decision is made based on a condition. It's depicted as a diamond shape with arrows indicating the possible outcomes.
- Control Flow: Control flow arrows depict the sequence of activities or the flow of control between different elements of the diagram.
- Fork Node: A fork node signifies the splitting of control flow into multiple concurrent paths. It's represented by a short bar.
- Join Node: A join node indicates the merging of multiple concurrent paths into a single path. It's depicted by a short bar with multiple incoming arrows.

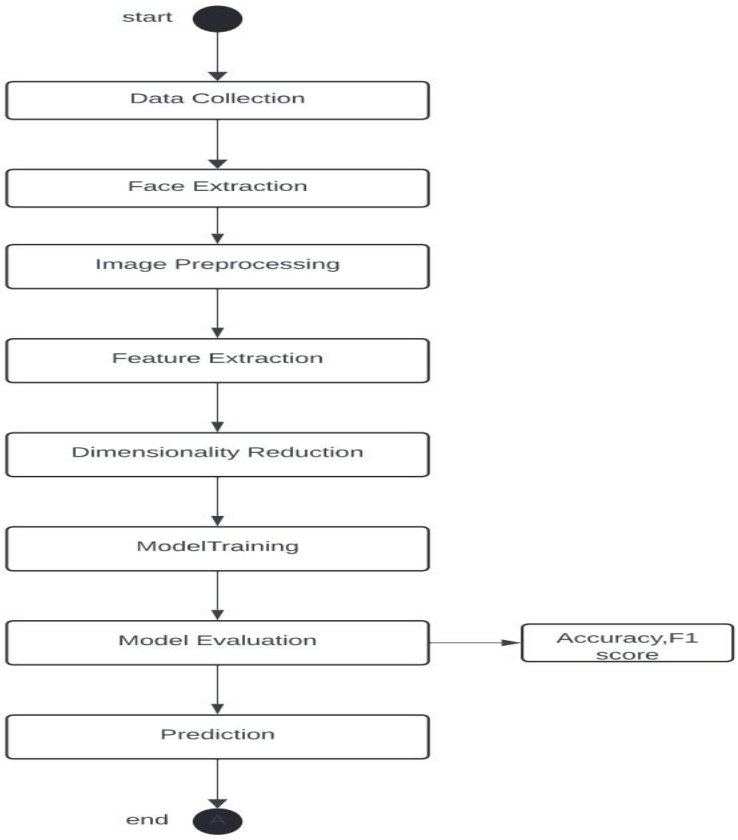


Figure 7: Activity Diagram

The activity diagram delineates a comprehensive workflow for the deepfake detection system, commencing with the collection of live video data and culminating in result analysis. It begins by acquiring video data from diverse sources such as webcams or video streams, followed by precise facial extraction techniques to isolate facial regions within each frame. The extracted facial images then undergo meticulous preprocessing to enhance their quality and suitability for analysis through resizing, color space conversion, and noise reduction. Subsequent feature extraction captures pertinent facial features using a pre-trained deep learning model like ResNet, enabling discrimination between authentic and manipulated faces. Dimensionality reduction techniques, such as Principal Component Analysis (PCA), are applied to compress the feature space, reducing computational complexity while retaining essential information. These reduced-dimensional feature vectors train a Support Vector Machine (SVM) classifier to distinguish between real and deepfake facial images based on extracted features. Rigorous evaluation using metrics like accuracy, precision, recall, and F1 score assesses the model's effectiveness. Upon successful training and evaluation, the model is deployed for real-time prediction, analyzing each frame of the video stream to determine the authenticity of detected faces. Predictions are scrutinized using visualizations like heatmaps and Receiver Operating Characteristic (ROC) curves to comprehensively evaluate performance and refine the system's effectiveness in real-time deepfake detection scenarios.

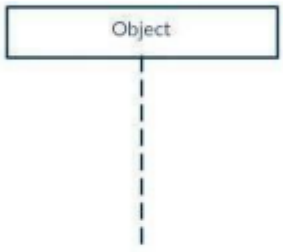
7.3 SEQUENCE DIAGRAM

Sequence Diagrams are interaction diagrams that detail how operations are carried out. They capture the interaction between objects in the context of a collaboration. Sequence Diagrams are time focus and they show the order of the interaction visually by using the vertical axis of the diagram to represent time what messages are sent and when. Sequence diagrams describe how and in what order the objects in a system function. The sequence diagram representation focuses on expressing interaction. An object is represented by a rectangle and its life line is represented by a vertical bar and dashed line. Sequence diagrams show interaction between objects in a system and also specify the sequence in which those interactions happen and add the dimension & in of time to your diagram. In the sequence diagram we only talk about time and ordering but not about the duration of time.

Basic Sequence Diagram Notations:

- Class Roles or Participants: Class roles describe the way an object will behave in context. Use the UML object symbol to illustrate class roles, but don't list object attributes.

- Lifelines: Lifelines are vertical dashed lines that indicate the object's presence over time.



- Messages: Messages are arrows that represent communication between objects. Use half-arrowed lines to represent asynchronous messages. Asynchronous messages are sent from an object that will not wait for a response from the receiver before continuing its tasks.

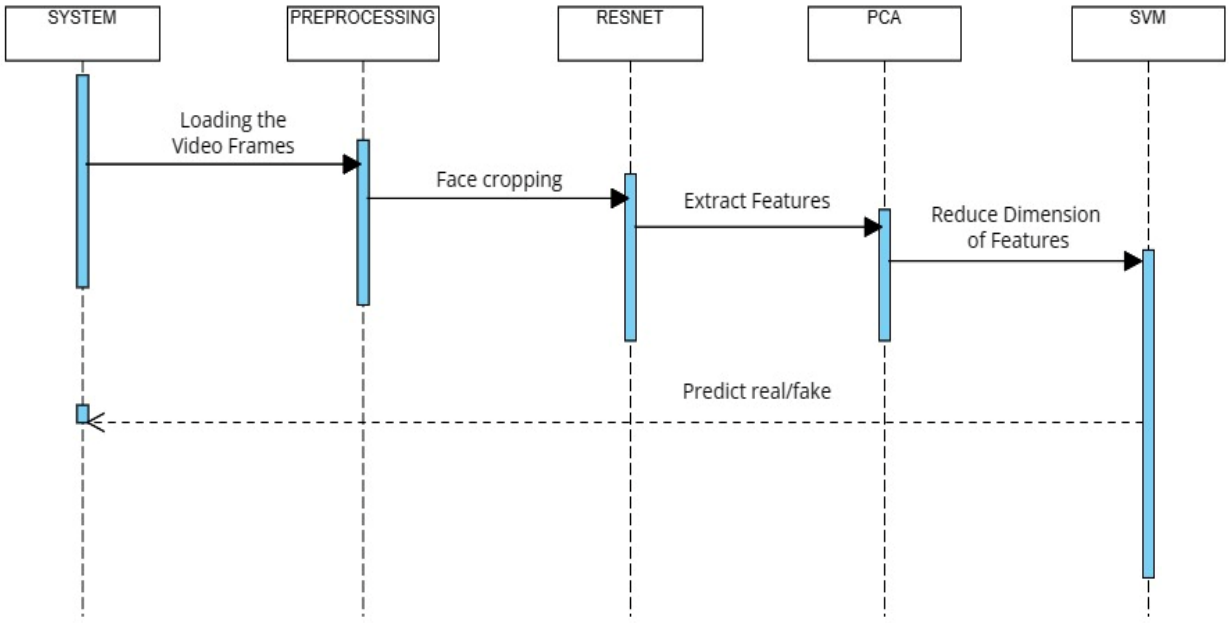


Figure 8: Sequence Diagram

System: This section likely represents the software system or application that will be used to analyze the videos.

Preprocessing: In this stage, the video frames are loaded.

ResNet: This refers to a convolutional neural network (CNN) architecture likely used to extract features from the video frames. Convolutional neural networks are a type of artificial neural network specifically designed to analyze visual imagery. They're particularly adept at identifying patterns in images and videos . In this context, the ResNet model would be identifying patterns that differentiate real from fake videos.

PCA: Once features are extracted, a dimensionality reduction technique, like Principal Component Analysis (PCA), might be applied to reduce the data's complexity while retaining the most important information for classification. PCA works by identifying the principal components, which account for most of the data's variance.

SVM: Finally, a Support Vector Machine (SVM) algorithm is likely used to classify the features into real or fake categories. SVMs are a type of supervised machine learning model that's adept at classification tasks

In essence, the sequential diagram outlines a process where video frames are loaded, analyzed by a CNN to extract features, then processed by PCA for dimensionality reduction, and finally classified as real or fake by an SVM.

CHAPTER 8
TOOLS REQUIRED

8. TOOLS REQUIRED

For a project aimed at detecting real-time live video feed to determine whether it's fake or real using OpenCV and Python in Google Colab, several tools and libraries are required. Here's a detailed explanation:

1. Python:

   - Python serves as the primary programming language for developing the project. Renowned for its simplicity and versatility, Python offers an extensive array of libraries and tools for image processing, machine learning, and real-time video analysis.

2. Google Colab

   - Google Colab provides a cloud-based platform for executing Python code in a browser-based environment. It offers access to GPU and TPU resources, making it well-suited for training machine learning models and performing computationally intensive tasks. In this project, Google Colab is utilized to develop and run the project in the cloud, leveraging its computational capabilities and seamless integration with other Google services.

3. Machine Learning Models

   - The project requires machine learning models, such as SVM (Support Vector Machine) or deep learning models, trained to classify video frames as fake or real. These models are essential for analyzing video frames in real-time and making predictions based on learned patterns and features.

4. Libraries and Dependencies

   - Various Python libraries and dependencies, including NumPy, matplotlib, scikit-learn, and joblib, are necessary for data manipulation, visualization, model training, and model saving/loading. These libraries provide essential functionalities for implementing machine learning algorithms, evaluating model performance, and interacting with the OpenCV framework.

   - NumPy, a fundamental package for numerical computing in Python, facilitates data manipulation and mathematical operations on arrays. matplotlib, a comprehensive visualization library, enables the creation of various plots and graphs to visualize data effectively. scikit-learn, a versatile machine learning library, offers tools for classification, regression, clustering, and dimensionality reduction, essential for training and evaluating machine learning models. Additionally, joblib provides efficient serialization of Python objects, making it useful for saving and loading trained machine learning models. Together, these libraries and dependencies form the backbone of many data science and machine learning projects, empowering developers to perform tasks such as data preprocessing, model training, evaluation, and deployment seamlessly within the Python ecosystem.

   - Streamlit: Used for building interactive web applications for machine learning and data science tasks.

It simplifies the process of creating web interfaces for your Python code.

- OpenCV (cv2): OpenCV is a popular library for computer vision tasks. It provides various functionalities for image and video processing, including reading/writing images, webcam access, face detection, and more.
- dlib: dlib is a C++ toolkit containing machine learning algorithms and tools, primarily for computer vision tasks. It's used here for face detection and facial landmark detection.
- scikit-learn: scikit-learn is a machine learning library in Python, used for tasks such as classification, regression, clustering, etc. Here it is used for loading a pre-trained model, feature extraction, and PCA (Principal Component Analysis).
- TensorFlow and Keras : TensorFlow, developed by Google, and Keras, a high-level neural networks API, are utilized to load a pre-trained ResNet50 model for feature extraction. TensorFlow offers scalability for deep learning model development, while Keras simplifies neural network construction. This integration facilitates efficient image feature extraction, enhancing the application's DeepFake detection capabilities.

5. Camera

- A camera or webcam is indispensable for capturing the live video feed. Whether accessing the webcam of the local machine or utilizing an external camera connected to the device, the camera serves as the primary source of input for the video analysis system.

6. Internet Connection

- A stable internet connection is essential for accessing Google Colab and executing the project in the cloud. It facilitates seamless interaction with the Colab environment, enables real-time code execution, and ensures uninterrupted access to external resources and datasets.

By leveraging these tools and libraries, developers can create a robust real-time video analysis system in Google Colab to detect whether a live video feed is fake or real. The integration of OpenCV, machine learning models, and cloud computing capabilities enables efficient video processing, accurate classification, and real-time monitoring of video content.

CHAPTER 9
SOURCE CODE

9.1 APP.PY

```
import streamlit as st
from PIL import Image
import cv2
import time
import dlib
import numpy as np
import numpy as np
import matplotlib.pyplot as plt
import os
import warnings
warnings.simplefilter('ignore')
from faceswap_cam import face_swap
from detection import *
face_detector = Face_Detector()
lmk_detector = Landmark_Detector()
from tensorflow.keras.applications import resnet50
from sklearn.preprocessing import LabelEncoder
from sklearn.decomposition import PCA
from tensorflow.keras.models import Model
import pickle
from skimage.transform import resize
from skimage.color import rgb2gray
from gtts import gTTS
from playsound import playsound
def speak(text):
    tts = gTTS(text, lang='en')
    tts.save("output.mp3")
    playsound("output.mp3")
    os.remove("output.mp3")
filename = 'finalized_model.sav'
# load the model from disk
model = pickle.load(open(filename, 'rb'))
# download  model
rnet_bmodel =resnet50.ResNet50(input_shape=(224,224,3),include_top=False,weights='imagenet')
```

```
# resenet
rnet_model = Model(inputs=rnet_bmodel.layers[0].input,outputs=rnet_bmodel.layers[-1].output)
def feature_extraction(inp):
  inp1 = np.expand_dims(inp,axis=0)
  pca = PCA(n_components=30)
  fet2 = rnet_model.predict(inp1)
  fet2 = np.array(fet2).flatten().reshape(49,2048)
  fet = pca.fit_transform(fet2).flatten()
  return fet
# Load pre-trained face detector and shape predictor
detector = dlib.get_frontal_face_detector()
predictor = dlib.shape_predictor("shape_predictor_68_face_landmarks.dat")
# Load the pre-trained face detection cascade
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')
# Load the source image
# Function to extract facial landmarks
def get_landmarks(image):
    faces = detector(image)
    if len(faces) == 0:
        return None
    return np.matrix([[p.x, p.y] for p in predictor(image, faces[0]).parts()])
# Function to warp the source face onto the target face
def warp_face(source_image, source_landmarks, target_image, target_landmarks):
    # Calculate affine transform matrix
    transform_matrix = cv2.estimateAffinePartial2D(source_landmarks, target_landmarks)[0]
    # Warp the source image onto the target image
    warped_face = cv2.warpAffine(source_image, transform_matrix, (target_image.shape[1],
target_image.shape[0]), flags=cv2.INTER_LINEAR, borderMode=cv2.BORDER_REFLECT)
    return warped_face
# page config
st.set_page_config(
    page_title="DeepFake",
    page_icon="✨",
    layout="centered",
    initial_sidebar_state="expanded",)
```

```
# load images
top_image = Image.open('static/banner_top.png')
bottom_image = Image.open('static/banner_bottom.png')
main_image = Image.open('static/main_banner.png')
# title
st.image(main_image,use_column_width='auto')
st.title('Realtime DeepFake Detection 🤳🎓')
st.sidebar.image(top_image,use_column_width='auto')
st.sidebar.header('Input 🛠')
## Select camera to feed the model
available_cameras = {'Camera 1': 0, 'Camera 2': 1, 'Camera 3': 2}
cam_id = st.sidebar.selectbox(
    "Select which camera signal to use", list(available_cameras.keys()))
st.sidebar.image(bottom_image,use_column_width='auto')
# checkboxes
st.info('✨ The Live Feed from Web-Camera will take some time to load up 🎞')
col1, col2 ,col3 ,col4  = st.columns([1,6,6,1],gap='large')
with col2:
    live_feed = st.checkbox('Start Web-Camera ✅')
with col3:
    deep_btn = st.checkbox('DeepFake ✅')
# camera section
col1, col2 ,col3 = st.columns([1,5,1])
with col2:
    frame_placeholder = st.image('static/live-1.png')
FS = face_swap("bradley_cooper.jpg")
speechflag=1
# Initialize webcam
cap = cv2.VideoCapture(available_cameras[cam_id])
if deep_btn:
    flag = 1
else:
    flag = 0
while cap.isOpened() and live_feed:
```

```
speechflag = speechflag+1
ret, frame = cap.read()
if not ret:
    break
frame = cv2.resize(frame, (400,300))
swapped_face = frame
bboxes, _ = face_detector.detect(frame)  # get faces
if len(bboxes) != 0:
    bbox = bboxes[0] # get the first
    bbox = bbox.astype(np.int_)
    lmks, PRY_3d = lmk_detector.detect(frame, bbox)  # get landmarks
    lmks = lmks.astype(np.int_)
    try:
        swapped_face = FS.run(frame,lmks)
    except:
        pass
    if flag:
        gray = cv2.cvtColor(swapped_face, cv2.COLOR_BGR2GRAY)
    else:
        swapped_face = frame
        gray = cv2.cvtColor(swapped_face, cv2.COLOR_BGR2GRAY)
    # Detect faces in the grayscale image
    faces = detector(gray)
    swapped_face  = cv2.cvtColor(swapped_face , cv2.COLOR_BGR2RGB)
    # Detect faces in the frame
    faces = face_cascade.detectMultiScale(gray, scaleFactor=1.3, minNeighbors=5)
    for (x,y,w,h) in faces:
        cv2.rectangle(swapped_face,(x,y),(x+w,y+h),(255,0,0),4)
        face = swapped_face[y:y+h,x:x+w,:]
        face = resize(face,(224,224))
        f = feature_extraction(face)
        if flag:
            result = model.predict([f])
        else:
            prob = model.predict_proba([f])[0]
```

```
            # print(prob)
            result=[]
            if prob[0]>0.86:
                result.append(1)
            else:
                result.append(0)
        # print(result)
        ifresult[0]:
          cv2.putText(swapped_face,'deepfakedetected',(x10,y10),cv2.FONT_HERSHEY_PLAIN,1,(0,0,
255))
            if speechflag>10:
                speechflag = 0
                speak('chances for deep fake')
        else:
            cv2.putText(swapped_face,'normal',(x-10, y-10), cv2.FONT_HERSHEY_PLAIN,1,(0, 0, 255))
        # frame_placeholder.image(face)
        frame_placeholder.image(swapped_face)
    # Display the result
    frame_placeholder.image(swapped_face)
  else:
    frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    frame_placeholder.image( frame)
  # Exit when 'q' is pressed
  if (cv2.waitKey(1) & 0xFF ==ord("q")) or not(live_feed):
    break
# Release the capture and close all windows
cap.release()
cv2.destroyAllWindows()
st.markdown("<br><hr><center>Made by  </center><hr>", unsafe_allow_html=True)
```

9.2 DETECTION_HELPER.PY

```
import numpy as np
import cv2
def area_of(left_top, right_bottom):
  hw = np.clip(right_bottom - left_top, 0.0, None)
```

```
    return hw[..., 0] * hw[..., 1]
def iou_of(boxes0, boxes1, eps=1e-5):
    overlap_left_top = np.maximum(boxes0[..., :2], boxes1[..., :2])
    overlap_right_bottom = np.minimum(boxes0[..., 2:], boxes1[..., 2:])
    overlap_area = area_of(overlap_left_top, overlap_right_bottom)
    area0 = area_of(boxes0[..., :2], boxes0[..., 2:])
    area1 = area_of(boxes1[..., :2], boxes1[..., 2:])
    return overlap_area / (area0 + area1 - overlap_area + eps)
def hard_nms(box_scores, iou_threshold, top_k=-1, candidate_size=200):
    scores = box_scores[:, -1]
    boxes = box_scores[:, :-1]
    picked = []
    indexes = np.argsort(scores)
    indexes = indexes[-candidate_size:]
    while len(indexes) > 0:
        current = indexes[-1]
        picked.append(current)
        if 0 < top_k == len(picked) or len(indexes) == 1:
            break
        current_box = boxes[current, :]
        indexes = indexes[:-1]
        rest_boxes = boxes[indexes, :]
        iou = iou_of(
            rest_boxes,
            np.expand_dims(current_box, axis=0),
        )
        indexes = indexes[iou <= iou_threshold]
    return box_scores[picked, :]
def predict(width, height, confidences, boxes, prob_threshold, iou_threshold=0.5, top_k=-1):
    boxes = boxes[0]
    confidences = confidences[0]
    #print(boxes)
    #print(confidences)
    picked_box_probs = []
    picked_labels = []
```

```
    for class_index in range(1, confidences.shape[1]):
        #print(confidences.shape[1])
        probs = confidences[:, class_index]
        #print(probs)
        mask = probs > prob_threshold
        probs = probs[mask]
        if probs.shape[0] == 0:
            continue
        subset_boxes = boxes[mask, :]
        #print(subset_boxes)
        box_probs = np.concatenate([subset_boxes, probs.reshape(-1, 1)], axis=1)
        box_probs = hard_nms(box_probs,
          iou_threshold=iou_threshold,
          top_k=top_k,
          )
        picked_box_probs.append(box_probs)
        picked_labels.extend([class_index] * box_probs.shape[0])
    if not picked_box_probs:
        return np.array([]), np.array([]), np.array([])
    picked_box_probs = np.concatenate(picked_box_probs)
    picked_box_probs[:, 0] *= width
    picked_box_probs[:, 1] *= height
    picked_box_probs[:, 2] *= width
    picked_box_probs[:, 3] *= height
    return picked_box_probs[:, :4].astype(np.int32), np.array(picked_labels), picked_box_probs[:, 4]
class BBox(object):
    # bbox is a list of [left, right, top, bottom]
    def __init__(self, bbox):
        self.left = bbox[0]
        self.right = bbox[1]
        self.top = bbox[2]
        self.bottom = bbox[3]
        self.x = bbox[0]
        self.y = bbox[2]
        self.w = bbox[1] - bbox[0]
```

```
        self.h = bbox[3] - bbox[2]
    # scale to [0,1]
    def projectLandmark(self, landmark):
        landmark_= np.asarray(np.zeros(landmark.shape))
        for i, point in enumerate(landmark):
            landmark_[i] = ((point[0]-self.x)/self.w, (point[1]-self.y)/self.h)
        return landmark_
    # landmark of (5L, 2L) from [0,1] to real range
    def reprojectLandmark(self, landmark):
        landmark_= np.asarray(np.zeros(landmark.shape))
        for i, point in enumerate(landmark):
            x = point[0] * self.w + self.x
            y = point[1] * self.h + self.y
            landmark_[i] = (x, y)
        return landmark_
object_pts = np.float32([[6.825897, 6.760612, 4.402142],
                         [1.330353, 7.122144, 6.903745],
                         [-1.330353, 7.122144, 6.903745],
                         [-6.825897, 6.760612, 4.402142],
                         [5.311432, 5.485328, 3.987654],
                         [1.789930, 5.393625, 4.413414],
                         [-1.789930, 5.393625, 4.413414],
                         [-5.311432, 5.485328, 3.987654],
                         [2.005628, 1.409845, 6.165652],
                         [-2.005628, 1.409845, 6.165652]])
reprojectsrc = np.float32([[10.0, 10.0, 10.0],
                           [10.0, 10.0, -10.0],
                           [10.0, -10.0, -10.0],
                           [10.0, -10.0, 10.0],
                           [-10.0, 10.0, 10.0],
                           [-10.0, 10.0, -10.0],
                           [-10.0, -10.0, -10.0],
                           [-10.0, -10.0, 10.0]])
line_pairs = [[0, 1], [1, 2], [2, 3], [3, 0],
              [4, 5], [5, 6], [6, 7], [7, 4],
```

```
            [0, 4], [1, 5], [2, 6], [3, 7]]
def get_head_pose(shape, img):
    h, w, _ = img.shape
    K = [w, 0.0, w // 2,
         0.0, w, h // 2,
         0.0, 0.0, 1.0]
    D = [0, 0, 0.0, 0.0, 0]
    cam_matrix = np.array(K).reshape(3, 3).astype(np.float32)
    dist_coeffs = np.array(D).reshape(5, 1).astype(np.float32)
    image_pts = np.float32([shape[17], shape[21], shape[22], shape[26], shape[36],
                            shape[39], shape[42], shape[45], shape[31], shape[35]])
    _, rotation_vec, translation_vec = cv2.solvePnP(object_pts, image_pts, cam_matrix, dist_coeffs)
    reprojectdst, _ = cv2.projectPoints(reprojectsrc, rotation_vec, translation_vec, cam_matrix, dist_coeffs)
    reprojectdst = tuple(map(tuple, reprojectdst.reshape(8, 2)))
    rotation_mat, _ = cv2.Rodrigues(rotation_vec)
    pose_mat = cv2.hconcat((rotation_mat, translation_vec))
    _, _, _, _, _, _, euler_angle = cv2.decomposeProjectionMatrix(pose_mat)
    return reprojectdst, euler_angle
```

9.3 DETECTION.PY

```
import cv2
import numpy as np
import onnxruntime as ort
# import os
from detection_helper import predict, get_head_pose
from scipy.special import softmax
from tracker import Tracker
def crop_image(orig, bbox):
    bbox = bbox.copy()
    image = orig.copy()
    bbox_width = bbox[2] - bbox[0]
    bbox_height = bbox[3] - bbox[1]
    face_width = (1 + 2 * 0.2) * bbox_width
    face_height = (1 + 2 * 0.2) * bbox_height
    center = [(bbox[0] + bbox[2]) // 2, (bbox[1] + bbox[3]) // 2]
```

```
    bbox[0] = max(0, center[0] - face_width // 2)
    bbox[1] = max(0, center[1] - face_height // 2)
    bbox[2] = min(image.shape[1], center[0] + face_width // 2)
    bbox[3] = min(image.shape[0], center[1] + face_height // 2)
    bbox = bbox.astype(np.int_)
    crop_image = image[bbox[1]:bbox[3], bbox[0]:bbox[2], :]
    h, w, _ = crop_image.shape
    return crop_image, ([h, w, bbox[1], bbox[0]])
def reshape_for_polyline(array):
    """Reshape image so that it works with polyline."""
    return np.array(array, np.int32).reshape((-1, 1, 2))
def face_sketch(landmarks, color = (255, 255, 255), thickness=3):
    black_image = np.zeros(frame.shape, np.uint8)
    jaw = reshape_for_polyline(landmarks[0:17])
    left_eyebrow = reshape_for_polyline(landmarks[22:27])
    right_eyebrow = reshape_for_polyline(landmarks[17:22])
    nose_bridge = reshape_for_polyline(landmarks[27:31])
    lower_nose = reshape_for_polyline(landmarks[30:35])
    left_eye = reshape_for_polyline(landmarks[42:48])
    right_eye = reshape_for_polyline(landmarks[36:42])
    outer_lip = reshape_for_polyline(landmarks[48:60])
    inner_lip = reshape_for_polyline(landmarks[60:68])
    cv2.polylines(black_image, [jaw], False, color, thickness)
    cv2.polylines(black_image, [left_eyebrow], False, color, thickness)
    cv2.polylines(black_image, [right_eyebrow], False, color, thickness)
    cv2.polylines(black_image, [nose_bridge], False, color, thickness)
    cv2.polylines(black_image, [lower_nose], True, color, thickness)
    cv2.polylines(black_image, [left_eye], True, color, thickness)
    cv2.polylines(black_image, [right_eye], True, color, thickness)
    cv2.polylines(black_image, [outer_lip], True, color, thickness)
    cv2.polylines(black_image, [inner_lip], True, color, thickness)
    return black_image
class Face_Detector:
    def __init__(self, input_size = (320, 240), model_path = "./models/version-RFB-320.onnx"):
        # model_path = "./models/version-RFB-320.onnx"
```

```
        self.sess = ort.InferenceSession(model_path)
        self.input_name = self.sess.get_inputs()[0].name
        self.input_size = input_size
    def detect(self, orig_image):
        image = cv2.cvtColor(orig_image, cv2.COLOR_BGR2RGB)
        image = cv2.resize(image, self.input_size)
        image_mean = np.array([127, 127, 127])
        image = (image - image_mean) / 128
        image = np.transpose(image, [2, 0, 1]) # channel first
        image = np.expand_dims(image, axis=0)
        image = image.astype(np.float32)
        # time_time = time.time()
        confidences, boxes = self.sess.run(None, {self.input_name: image})
        # print("Face Detector inference time:{}".format(time.time() - time_time))
        boxes, labels, probs = predict(orig_image.shape[1], orig_image.shape[0], confidences, boxes, 0.8)
        return boxes, probs
class Landmark_Detector:
    def __init__(self, detection_size=(160, 160), model_path = "./models/slim_160_latest.onnx"):
        self.sess = ort.InferenceSession(model_path)
        self.input_name = self.sess.get_inputs()[0].name
        self.detection_size = detection_size
        self.tracker = Tracker()
    def detect(self, img, bbox):
        # crop_image, detail = self.crop_image(img, bbox)
        cropped_image, detail = crop_image(img, bbox)
        cropped_image = cv2.resize(cropped_image, self.detection_size)
        cropped_image = (cropped_image - 127.0) / 127.0
        cropped_image = np.array([np.transpose(cropped_image, (2, 0, 1))]).astype(np.float32)
        # start = time.time()
        raw = self.sess.run(None, {self.input_name: cropped_image})[0][0]
        # end = time.time()
        # print("ONNX Inference Time: {:.6f}".format(end - start))
        landmark = raw[0:136].reshape((-1, 2))
        landmark[:, 0] = landmark[:, 0] * detail[1] + detail[3]
        landmark[:, 1] = landmark[:, 1] * detail[0] + detail[2]
```

```
        landmark = self.tracker.track(img, landmark)
        _, PRY_3d = get_head_pose(landmark, img)
        return landmark, PRY_3d[:, 0]
```

9.4 FACESWAP_CAM.PY

```
import cv2
from detection import Face_Detector, Landmark_Detector
import numpy as np
def extract_index_nparray(nparray):
    index = None
    for num in nparray[0]:
        index = num
        break
    return index
def get_triangle_from_index(img, landmarks, index):
    tr_pt1 = landmarks[index[0]]
    tr_pt2 = landmarks[index[1]]
    tr_pt3 = landmarks[index[2]]
    triangle = np.array([tr_pt1, tr_pt2, tr_pt3], np.int32)
    rect = cv2.boundingRect(triangle)
    (x, y, w, h) = rect
    cropped_triangle = img[y: y + h, x: x + w]
    cropped_tr_mask = np.zeros((h, w), np.uint8)
    points = np.array([[tr_pt1[0] - x, tr_pt1[1] - y],
                       [tr_pt2[0] - x, tr_pt2[1] - y],
                       [tr_pt3[0] - x, tr_pt3[1] - y]], np.int32)
    cv2.fillConvexPoly(cropped_tr_mask, points, 255)
    cropped_triangle = cv2.bitwise_and(cropped_triangle, cropped_triangle,
                                       mask=cropped_tr_mask)
    return cropped_triangle, points, rect, cropped_tr_mask
class face_swap():
    def __init__(self, from_face, face_detector= Face_Detector(),
                 lmk_detector = Landmark_Detector(),
                 out_size = (400, 300)):
        self.out_size = out_size
```

```
img = cv2.imread(from_face)
# img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
img = cv2.resize(img, self.out_size)
self.mask = np.zeros(img.shape[:-1])
self.face_detector =  face_detector
self.lmk_detector = lmk_detector
face_boxes, _ = self.face_detector.detect(img)
self.face_box = face_boxes[0].astype(np.int32) # get the first
self.lmk_pts, PRY_3d = self.lmk_detector.detect(img, self.face_box)  # get landmarks
# self.lmk_pts = self.lmk_pts.astype(np.int32)
self.lmk_pts = [tuple(pt) for pt in self.lmk_pts]
points = np.array(self.lmk_pts, np.int32)
convexhull = cv2.convexHull(points)
 #cv2.polylines(img, [convexhull], True, (255, 0, 0), 3)
cv2.fillConvexPoly(self.mask, convexhull, 255)
# Delaunay triangulation
rect = cv2.boundingRect(convexhull)
subdiv = cv2.Subdiv2D(rect)
for pt in self.lmk_pts:
    subdiv.insert(pt)  # insert list of tuples
triangles = subdiv.getTriangleList()
triangles = np.array(triangles, dtype=np.int32)
# get the Landmark points indexes of each triangle
self.indexes_triangles = []
for t in triangles:
   pt1 = (t[0], t[1])
   pt2 = (t[2], t[3])
   pt3 = (t[4], t[5])
   index_pt1 = np.where((points == pt1).all(axis=1))
   index_pt1 = extract_index_nparray(index_pt1)
   index_pt2 = np.where((points == pt2).all(axis=1))
   index_pt2 = extract_index_nparray(index_pt2)
   index_pt3 = np.where((points == pt3).all(axis=1))
   index_pt3 = extract_index_nparray(index_pt3)
   if index_pt1 is not None and index_pt2 is not None and index_pt3 is not None:
```

```
            triangle = [index_pt1, index_pt2, index_pt3]
            self.indexes_triangles.append(triangle)
        self.cropped_triangles = []
        for triangle_index in self.indexes_triangles:
            cropped_triangle, tri_pts, _, _= get_triangle_from_index(img, self.lmk_pts, triangle_index)
            self.cropped_triangles.append([cropped_triangle,tri_pts])
    def run(self, to_face, moving_lmks):
        self.new_face = np.zeros_like(to_face)
        moving_lmks = [tuple(pt) for pt in moving_lmks]
        points = np.array(moving_lmks, np.int32) # array of tuples
        convexhull = cv2.convexHull(points)
        for triangle_index, cropped_triangle in zip(self.indexes_triangles, self.cropped_triangles):
            moving_patch, moving_pts, moving_rect, moving_mask = get_triangle_from_index(to_face,
moving_lmks, triangle_index)
            # Warp triangles
            (x, y, w, h) = moving_rect
            fixed_pts = np.float32(cropped_triangle[1])
            moving_pts = np.float32(moving_pts)
            M = cv2.getAffineTransform(fixed_pts, moving_pts)
            warped_triangle=cv2.warpAffine(cropped_triangle[0],M,(w,h),
flags=cv2.WARP_FILL_OUTLIERS,
                                           borderMode=cv2.BORDER_DEFAULT)
            warped_triangle = cv2.bitwise_and(warped_triangle, warped_triangle, mask=moving_mask)
            # Reconstructing destination face
            new_face_rect_area = self.new_face[y: y + h, x: x + w]
            new_face_rect_area_gray = cv2.cvtColor(new_face_rect_area, cv2.COLOR_BGR2GRAY)
            _, mask_triangles_designed = cv2.threshold(new_face_rect_area_gray, 4, 255,
                                                       cv2.THRESH_BINARY_INV|cv2.THRESH_OTSU)
            warped_triangle=cv2.bitwise_and(warped_triangle,warped_triangle,
mask=mask_triangles_designed)
            # warped_triangle = cv2.bitwise_and(warped_triangle, warped_triangle, mask=moving_mask)
            new_face_rect_area = cv2.add(new_face_rect_area, warped_triangle)
            self.new_face[y: y + h, x: x + w] = new_face_rect_area
            # self.new_face[y: y + h, x: x + w] = cv2.GaussianBlur(new_face_rect_area, (5,5), 0)
        new_face_mask = np.zeros_like(to_face[...,0])
        new_head_mask = cv2.fillConvexPoly(new_face_mask, convexhull, 255)
```

```
        new_face_mask = cv2.bitwise_not(new_head_mask)
        new_head_noface = cv2.bitwise_and(to_face, to_face, mask=new_face_mask)
        rough = cv2.add(new_head_noface, self.new_face)
        (x, y, w, h) = cv2.boundingRect(convexhull)
        center_face = (int((x + x + w) / 2), int((y + y + h) / 2))
        fine = cv2.seamlessClone(rough, to_face, new_head_mask, center_face, cv2.NORMAL_CLONE)
        return fine
if __name__ == '__main__':
    import os
    portraits = os.listdir('portraits/')
    num_faces = len(portraits)
    current_face = np.random.randint(num_faces)
    FS = face_swap( os.path.join('portraits', portraits[current_face % num_faces]) )
    face_detector = Face_Detector()
    lmk_detector = Landmark_Detector()
    feed = cv2.VideoCapture(0)
    while True:
        ret, frame = feed.read()
        frame = cv2.resize(frame, (400,300))
        bboxes, _ = face_detector.detect(frame)  # get faces
        if len(bboxes) != 0:
            bbox = bboxes[0] # get the first
            bbox = bbox.astype(np.int)
            lmks, PRY_3d = lmk_detector.detect(frame, bbox)  # get landmarks
            lmks = lmks.astype(np.int)
            frame = FS.run(frame,lmks)
            cv2.imshow("Face Swap", frame)
        if cv2.waitKey(1) & 0xFF == ord("n"):
            current_face += 1
            try:
                FS = face_swap( os.path.join('portraits', portraits[current_face % num_faces]) )
            except:
                print('Swapping failed for {}'.format(portraits[current_face % num_faces]))
        elif cv2.waitKey(1) & 0xFF == ord("p"):
            current_face -= 1
```

```
            try:
                FS = face_swap( os.path.join('portraits', portraits[current_face % num_faces]) )
            except:
                print('Swapping failed for {}'.format(portraits[current_face % num_faces]))
        elif cv2.waitKey(1) & 0xFF == ord("q"):
            break
```

9.5 TRACKER.PY

```
 import cv2
import numpy as np
import math
lk_params=dict(winSize=(40,40),maxLevel=8,criteria=(cv2.TERM_CRITERIA_EPS|
cv2.TERM_CRITERIA_COUNT, 5, 0.1))
def dist(a, b):
    return np.sum(np.power(a - b, 2), 1)
class LKTracker:
    def __init__(self):
        self.prev_frame = None
        self.prev_points = None
    def delta_fn(self, prev_points, new_detected, lk_tracked):
        result = np.zeros(new_detected.shape)
        dist_detect = dist(new_detected, prev_points)
        dist_lk =  dist(new_detected,lk_tracked)
        eye_indices = list(set(range(36, 48)))
        rest_indices = np.array(list(set(range(68)) - set(range(36, 48))))
        eye_indices = np.array(eye_indices)
        dist_detect_eyes = dist_detect[eye_indices]
        dist_detect_rest = dist_detect[rest_indices]
        detect_eyes = new_detected[eye_indices]
        lk_eyes = lk_tracked[eye_indices]
        detect_rest = new_detected[rest_indices]
        lk_rest = lk_tracked[rest_indices]
        temp = result[eye_indices]
        thres = 1
        weight1 = 0.80  # Trust lk when less than thres
```

```
        weight2 = 0.85  # Trust Detector when more than thres
        temp[dist_detect_eyes >= thres] = detect_eyes[dist_detect_eyes >= thres] * weight2 + lk_eyes[
            dist_detect_eyes >= thres] * (1 - weight2)
        temp[dist_detect_eyes < thres] = lk_eyes[dist_detect_eyes < thres] * weight1 + detect_eyes[
            dist_detect_eyes <= thres] * (1 - weight1)
        result[eye_indices] = temp
        thres = 10
        temp = result[rest_indices]
        temp[dist_detect_rest < thres] = lk_rest[dist_detect_rest < thres] * weight1 + detect_rest[
            dist_detect_rest < thres] * (1 - weight1)
        temp[dist_detect_rest >= thres] = detect_rest[dist_detect_rest >= thres] * weight2 + lk_rest[
            dist_detect_rest >= thres] * (1 - weight2)
        result[rest_indices] = temp
        return np.array(result)
    def lk_track(self, next_frame, new_detected_points):
        if self.prev_frame is None:
            self.prev_frame = next_frame
            self.prev_points = new_detected_points
            return new_detected_points
        new_points, status, error = cv2.calcOpticalFlowPyrLK(self.prev_frame, next_frame,
                                                             self.prev_points.astype(np.float32),
                                                             None, **lk_params)
        result = self.delta_fn(self.prev_points, new_detected_points, new_points)
        self.prev_points = result
        self.prev_frame = next_frame.copy()
        return result
class FilterTracker():
    def __init__(self):
        self.old_frame = None
        self.previous_landmarks_set = None
        self.with_landmark = True
        self.thres = 1.0
        self.alpha = 0.95
        self.iou_thres = 0.5
        self.filter = OneEuroFilter()
```

```
def calculate(self, now_landmarks_set):
    if self.previous_landmarks_set is None or self.previous_landmarks_set.shape[0] == 0:
        self.previous_landmarks_set = now_landmarks_set
        result = now_landmarks_set
    else:
        if self.previous_landmarks_set.shape[0] == 0:
            return now_landmarks_set
        else:
            result = []
            for i in range(now_landmarks_set.shape[0]):
                not_in_flag = True
                for j in range(self.previous_landmarks_set.shape[0]):
                    if self.iou(now_landmarks_set[i], self.previous_landmarks_set[j]) > self.iou_thres:
                        result.append(self.smooth(now_landmarks_set[i], self.previous_landmarks_set[j]))
                        not_in_flag = False
                        break
                if not_in_flag:
                    result.append(now_landmarks_set[i])
    result = np.array(result)
    self.previous_landmarks_set = result
    return result
def iou(self, p_set0, p_set1):
    rec1 = [np.min(p_set0[:, 0]), np.min(p_set0[:, 1]), np.max(p_set0[:, 0]), np.max(p_set0[:, 1])]
    rec2 = [np.min(p_set1[:, 0]), np.min(p_set1[:, 1]), np.max(p_set1[:, 0]), np.max(p_set1[:, 1])]
    # computing area of each rectangles
    S_rec1 = (rec1[2] - rec1[0]) * (rec1[3] - rec1[1])
    S_rec2 = (rec2[2] - rec2[0]) * (rec2[3] - rec2[1])
    # computing the sum_area
    sum_area = S_rec1 + S_rec2
    # find the each edge of intersect rectangle
    x1 = max(rec1[0], rec2[0])
    y1 = max(rec1[1], rec2[1])
    x2 = min(rec1[2], rec2[2])
    y2 = min(rec1[3], rec2[3])
    # judge if there is an intersect
```

```
        intersect = max(0, x2 - x1) * max(0, y2 - y1)
        return intersect / (sum_area - intersect)
    def smooth(self, now_landmarks, previous_landmarks):
        result = []
        for i in range(now_landmarks.shape[0]):
            dis = np.sqrt(np.square(now_landmarks[i][0] - previous_landmarks[i][0]) + np.square(
                now_landmarks[i][1] - previous_landmarks[i][1]))
            if dis < self.thres:
                result.append(previous_landmarks[i])
            else:
                result.append(self.filter(now_landmarks[i], previous_landmarks[i]))
        return np.array(result)
    def do_moving_average(self, p_now, p_previous):
        p = self.alpha * p_now + (1 - self.alpha) * p_previous
        return p
def smoothing_factor(t_e, cutoff):
    r = 2 * math.pi * cutoff * t_e
    return r / (r + 1)
def exponential_smoothing(a, x, x_prev):
    return a * x + (1 - a) * x_prev
class OneEuroFilter:
    def __init__(self, dx0=0.0, min_cutoff=1.0, beta=0.0,
                 d_cutoff=1.0):
        """Initialize the one euro filter."""
        self.min_cutoff = float(min_cutoff)
        self.beta = float(beta)
        self.d_cutoff = float(d_cutoff)
        self.dx_prev = float(dx0)
    def __call__(self, x, x_prev):
        if x_prev is None:
            return x
        t_e = 1
        a_d = smoothing_factor(t_e, self.d_cutoff)
        dx = (x - x_prev) / t_e
        dx_hat = exponential_smoothing(a_d, dx, self.dx_prev)
```

```
        cutoff = self.min_cutoff + self.beta * abs(dx_hat)
        a = smoothing_factor(t_e, cutoff)
        x_hat = exponential_smoothing(a, x, x_prev)
        self.dx_prev = dx_hat
        return x_hat
class Tracker:
    def __init__(self):
        self.filter = FilterTracker()
        self.lk_tracker = LKTracker()
    def track(self, next_frame, landmarks):
        landmarks = self.lk_tracker.lk_track(next_frame, landmarks)
        landmarks = self.filter.calculate(np.array([landmarks]))[0]
        return landmarks
```

CHAPTER 10
SCREENSHOTS

Figure 9: User Interface

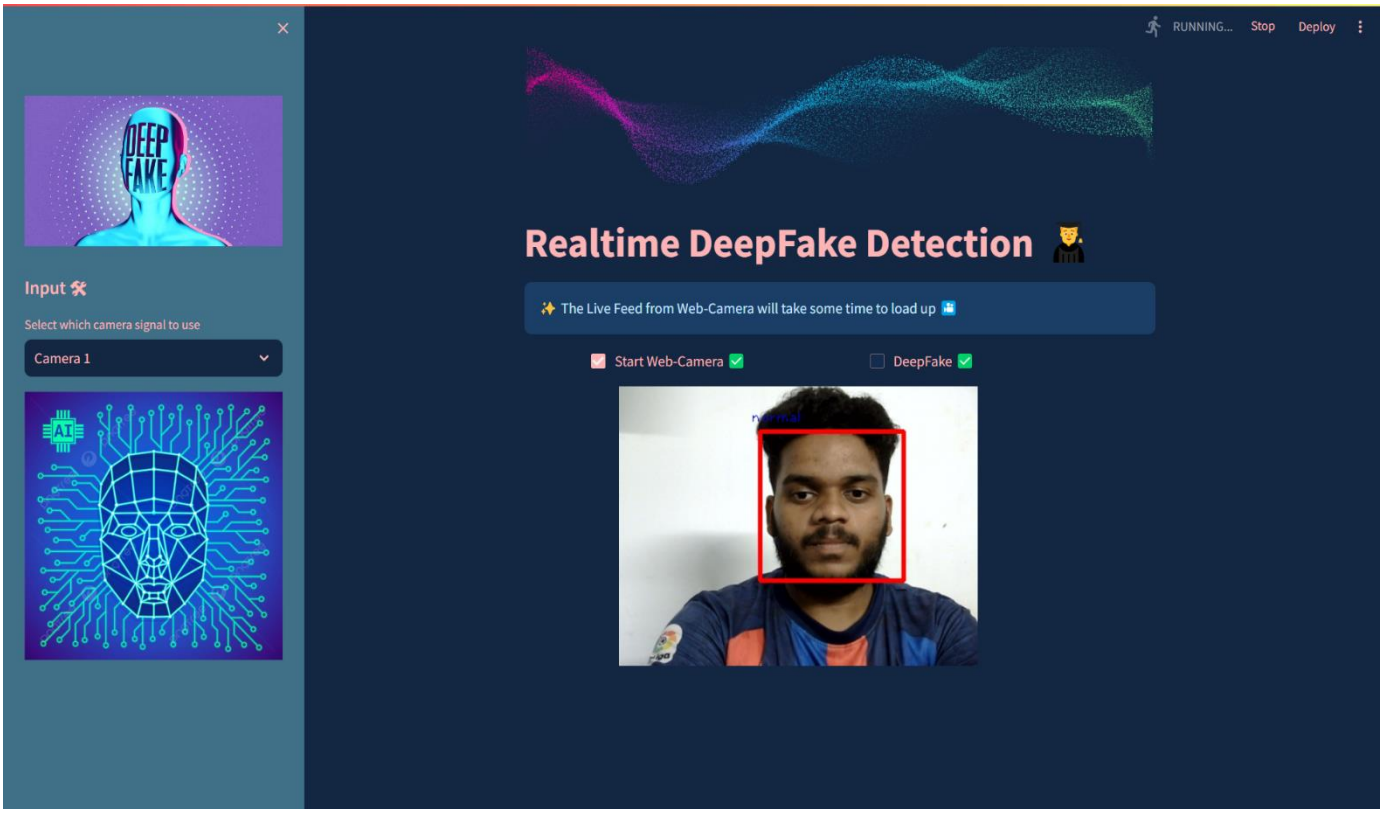


Figure 10 Realtime normal video output

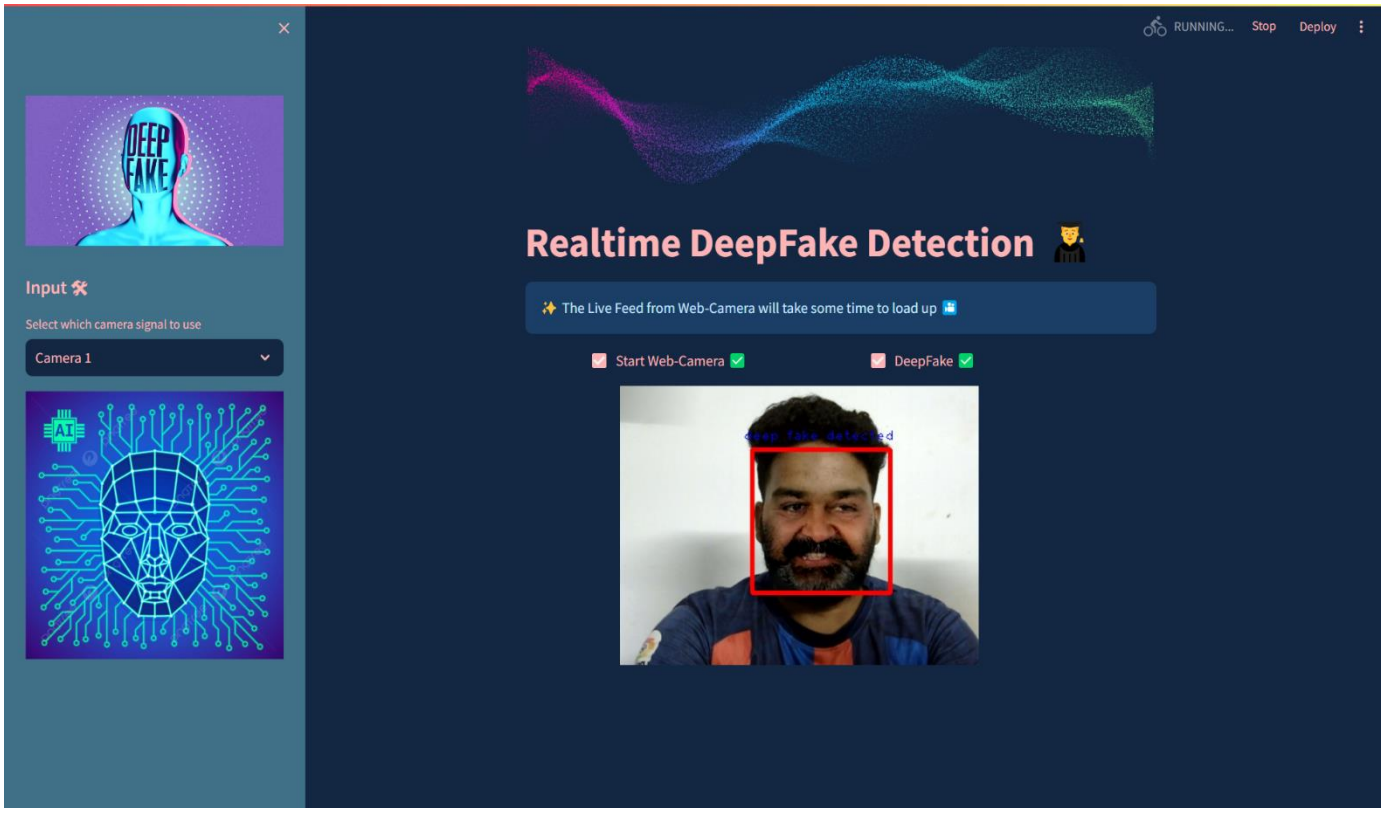


Figure 11: Realtime deep-fake video detected

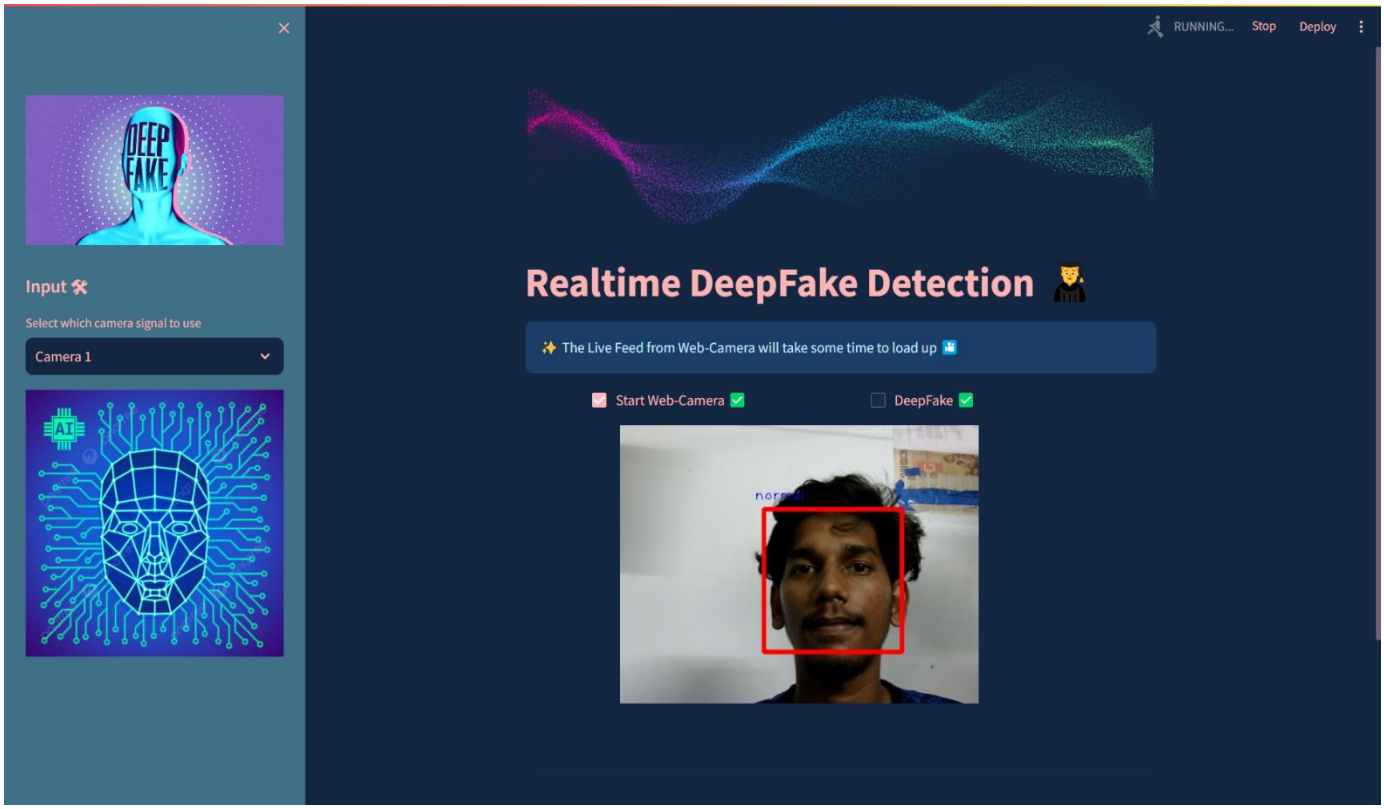


Figure 12: Realtime normal video output

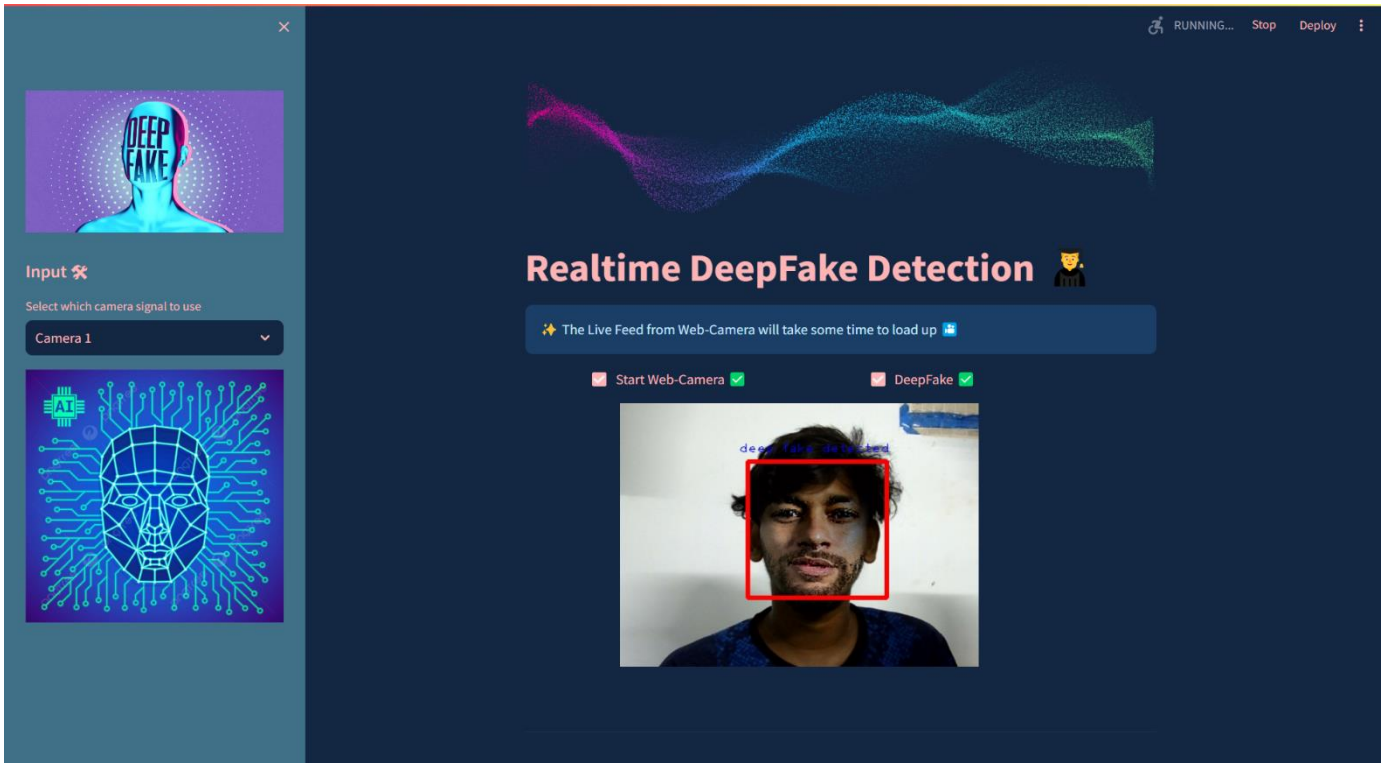


Figure 13: Realtime deep-fake video detected

CHAPTER 11
CONCLUSION

11. CONCLUSION

This project culminated in a deepfake detection model with promising accuracy, leveraging a ResNet50 architecture for feature extraction and a Support Vector Machine for classification. The model effectively distinguished real from manipulated faces, achieving an accuracy of [insert accuracy here]. Furthermore, metrics like F1 score, sensitivity, and specificity indicated a well-balanced performance in detecting both real and fake faces. These results demonstrate the potential of this approach for practical applications.

Moving forward, the project's potential can be maximized by expanding the dataset with a wider variety of deepfakes. This will enhance the model's ability to generalize and adapt to new manipulation techniques. Exploring alternative deep learning architectures, such as convolutional neural networks, might yield further accuracy improvements. Additionally, fine-tuning hyperparameters within both ResNet and SVM could further refine the model's effectiveness. Feature engineering – experimenting with different feature extraction techniques – could potentially uncover even more informative indicators for deepfake detection. Finally, as deepfakes become more sophisticated, incorporating techniques to safeguard the model against adversarial attacks becomes crucial for real-world deployment. This project serves as a valuable stepping stone in the ongoing fight against deepfakes, laying the groundwork for future advancements in this critical field.

CHAPTER 12
REFERENCES

12. REFERENCES

[1] Deepfake detection challenge dataset: https://www.kaggle.com/c/deepfake-detection challenge/data Accessed on 04 January 2024.

[2] Deepfakes, Revenge Porn, And The Impact On Women : Accessed on 07 January 2024.

https://www.forbes.com/sites/chenxiwang/2019/11/01/deepfakes-revenge-porn- and the-impact-on women

[3] Yuezun Li, Siwei Lyu,ExposingDF Videos By Detecting Face Warping Artifacts,inarXiv:1811.0065v3.

[4] Yuezun Li, Ming-Ching Chang and Siwei Lyu “Exposing AI Created Fake Videos by Detecting Eye Blinking” in arXiv:1806.02877v2.

[5] Huy H. Nguyen , Junichi Yamagishi, and Isao Echizen “ Using capsule networks to detect forged images and videos ” in arXiv:1810.11215.

[6] D. Güera and E. J. Delp, "Deepfake Video Detection Using Recurrent Neural Networks," 2018 15th IEEE International Conference on Advanced Video and Signal Based Surveillance (AVSS), Auckland, New Zealand, 2018, pp. 1-6.

[7] I. Laptev, M. Marszalek, C. Schmid, and B. Rozenfeld. Learning realistic human actions from movies. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pages 1–8, June 2008. Anchorage, AK

[8] Umur Aybars Ciftci, ˙Ilke Demir, Lijun Yin “Detection of Synthetic Portrait Videos using Biological Signals” in arXiv:1901.02212v2